A D PATEL INSTITUTE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

DUPLICATE PHOTO AND FILE FINDER

A Mini Project Report

A report submitted

In partial fulfilment of the requirements for the Degree of

Bachelor of Technology

In

Information Technology

Semester – V

Subject:

PROGRAMMING WITH PYTHON(202044504)

By

Raj Aniruddhsinh N (12402080601004)

Faculty Guide

Nayan Mali

Assistant Professor

Academic Year 2026–27

Department of Information Technology

A D Patel Institute of Technology

New Vallabh Vidyanagar, Vitthal Udyognagar – 388121 Anand (Gujarat), India

CERTIFICATE

This is to certify that the mini project entitled “Duplicate File Finder” has been developed and submitted by Raj Aniruddh N. of A.D. Patel Institute of Technology, CVM University, as part of the academic requirements for the subject Python Programming.
The project demonstrates the application of Python programming concepts, file handling, functions, exception handling, hashing, and graphical user interface development.
The work has been completed under the guidance of the concerned faculty member.

Date: _____

Place: New Vallabh Vidyanagar

Lab Teacher	Head of Department
Nayan Mali	**Dr. Priyang Bhatt**
Assistant Professor	Associate Professor & Head

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my Python Programming faculty for providing me with the opportunity to develop this mini project, “Duplicate File Finder.”
This project helped me understand how Python can be used to solve practical problems related to file management and storage optimization.
I am thankful to A.D. Patel Institute of Technology and CVM University for providing the academic environment and resources required to complete this project.
Finally, I would like to thank everyone who supported and guided me during the development of this project.

Raj Aniruddh
(12402080601004)

TABLE OF CONTENTS

Chapter	Title
1	Introduction
2	Problem Statement and Objectives
3	Technologies Used
4	System Analysis and Design
5	Algorithm and Methodology
6	Implementation
7	Testing and Results
8	Advantages, Limitations and Future Scope
9	Conclusion
—	References

LIST OF TABLES

Table 1: Comparison between different duplicate file detection methods
Table 2: Technologies used in the project
Table 3: Functional requirements of the application
Table 4: Time and space complexity analysis
Table 5: Test cases for the application
Table 6: Experimental results of the application

ABSTRACT

The Duplicate File Finder is a Python-based desktop application designed to identify duplicate files stored in a selected folder. Duplicate files may occupy unnecessary storage space and make file management difficult.

The application scans the selected folder and its subfolders, collects file information, and compares files to identify duplicates. It first compares file sizes to reduce unnecessary processing. Files with matching sizes are then compared using the SHA-256 hashing algorithm. Files with matching hashes are treated as duplicate candidates.

The application provides a graphical user interface developed using Python's Tkinter library. Users can select a folder, start scanning, view duplicate groups, export a report in CSV format, and move selected duplicates into a quarantine folder instead of permanently deleting them.

The project demonstrates practical applications of Python programming, file handling, functions, data structures, hashing, exception handling, and graphical user interface development.

Keywords: Python, Duplicate Files, File Handling, SHA-256, Tkinter, Hashing, CSV.

CHAPTER 1: INTRODUCTION

1.1 Introduction

Python is a high-level programming language known for its simple syntax, readability, and extensive standard library. It is widely used for automation, data processing, application development, and file management.

When users download files multiple times, copy folders, or create backups, the same file may be stored in different locations. These duplicate files consume storage space and make it difficult to organize data.

The Duplicate File Finder project is developed to solve this problem. It scans a selected directory and identifies files that contain identical data. The application presents the duplicate files to the user and provides options to export the results or move selected files into quarantine.

The project uses Python's built-in modules, so external libraries are not required.

1.2 Background

Manual identification of duplicate files is difficult when a computer contains hundreds or thousands of files. Files may have different names but contain exactly the same data.

For example, a photograph named photo.jpg may also exist as photo_copy.jpg in another folder. Comparing filenames alone will not identify these files as duplicates.

Therefore, the application uses file size and SHA-256 hashing to compare file contents more reliably.

1.3 Purpose of the Project

The main purpose of this project is to automate duplicate file detection and help users manage their storage efficiently.

The application reduces the effort required to find duplicate files manually and provides a safer alternative to immediately deleting files.

1.4 Scope of the Project

The project includes:

- Selecting a folder for scanning.
- Searching files in subfolders.
- Collecting file paths and sizes.

- **Comparing files using SHA-256 hashes.**
- **Displaying duplicate file groups.**
- **Exporting scan results to a CSV file.**
- **Moving selected files to a quarantine folder.**
- **Handling errors during file access.**

The project is designed as a local desktop application and does not require an internet connection.

CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Duplicate files are common in personal computers and storage devices. They may be created through repeated downloads, copying files, transferring data, or maintaining multiple backups.

Finding these files manually is time-consuming and inefficient. Comparing only filenames is not sufficient because duplicate files can have different names.

Therefore, a Python-based application is required to scan directories and identify duplicate files based on their contents.

2.2 Objectives

The objectives of the project are:

1. To develop a desktop application using Python.
2. To scan a selected directory and its subdirectories.
3. To identify duplicate files using file size and SHA-256 hashing.
4. To reduce unnecessary comparisons during scanning.
5. To display duplicate files in an understandable format.
6. To export duplicate information into a CSV report.
7. To provide a quarantine option for safer file management.
8. To handle file access errors without unnecessarily stopping the entire scan.

9. To apply Python programming concepts to a real-world problem.

2.3 Existing System

In the existing manual approach, users search folders and compare files individually. They may check filenames, extensions, sizes, or open files to determine whether their contents are identical.

This approach has several disadvantages:

- It requires considerable time.
- It becomes difficult with large numbers of files.
- Files with different names may be overlooked.
- Manual comparisons are repetitive.
- There is a greater possibility of human error.

2.4 Proposed System

The proposed system automates the process through a Python desktop application.

The user selects a folder and starts scanning. The application examines files, compares their sizes, calculates hashes for suitable candidates, and groups files with matching content.

The results are displayed in the interface, and the user can export the report or move selected files into quarantine.

2.5 Functional Requirements

The functional requirements describe what the application should do.

- Allow the user to select a folder.
- Scan files and subfolders.
- Calculate file sizes.
- Generate SHA-256 hashes.
- Identify and group duplicate files.
- Display scan progress and results.
- Export duplicate information to CSV.
- Quarantine selected duplicate files.
- Report errors when files cannot be accessed.

2.6 Non-Functional Requirements

- Usability: The interface should be simple to understand.
- Reliability: The application should handle common file-access errors.
- Performance: Size filtering should reduce unnecessary hashing.
- Safety: Selected files should be quarantined rather than immediately deleted.
- Portability: The application should run on systems that support Python and Tkinter.

CHAPTER 3: TECHNOLOGIES USED

3.1 Python Programming Language

Python is the primary programming language used to develop the application. Its standard library provides tools for file operations, hashing, threading, CSV processing, and graphical interface development.

3.2 Tkinter

Tkinter is Python's standard library for developing graphical user interfaces.

It provides components such as buttons, labels, tables or list displays, file selection dialogs, and progress indicators.

In this project, Tkinter allows users to operate the application without needing to type commands into a terminal.

3.3 OS and File-System Operations

Python's file-system functions are used to access folders, list files, obtain file sizes, and manage file paths.

These operations allow the application to scan directories and inspect the files they contain.

3.4 SHA-256 Hashing

SHA-256 is a cryptographic hash function that converts file data into a fixed-length hash value.

The application calculates a hash for files of the same size. If two files have the same SHA-256 hash, they are treated as duplicate candidates.

Hash comparison is more reliable than comparing filenames because the hash is based on the file contents.

3.5 CSV Module

CSV stands for Comma-Separated Values. It is a simple format for storing tabular information.

The application uses CSV export to save duplicate file paths and related information. The exported file can be opened in spreadsheet applications such as Microsoft Excel.

3.6 Threading and Queue

Scanning a large directory can take time. If scanning were performed directly in the main graphical interface thread, the interface could become unresponsive.

Threading allows scanning to take place in a background worker thread. A queue is used to communicate progress and results to the graphical interface.

3.7 Technologies Summary

Technology	Purpose
Python	Main programming language
Tkinter	Graphical user interface
File-system operations	Folder scanning and file information
`hashlib`	SHA-256 hash calculation
`csv`	Exporting scan results
`threading`	Background scanning
`queue`	Communication between the worker and interface
`os` and `pathlib`	File and directory management

CHAPTER 4: ALGORITHM AND COMPLEXITY ANALYSIS

4.1 Algorithm

The Duplicate Photo and File Finder uses file size and SHA-256 hashing to identify duplicate files.

Steps:

1. Select a folder and scan its files.
2. Collect file names, paths, and sizes.
3. Group files having the same size.
4. Calculate SHA-256 hashes for candidate files.
5. Compare hashes and identify duplicate groups.

6. Display the results.

4.2 Pseudocode

START

Select folder

Scan all files

Group files by size

FOR each group with multiple files

Calculate SHA-256 hash

Group files with matching hashes

END FOR

Display duplicate groups

STOP

4.3 Flowchart

Figure 4.1: Duplicate File Detection

Start

Select Folder and Scan Files

Group by File Size

Calculate and Compare Hashes

Display Duplicates

Stop

4.4 Complexity Analysis

Let \(n\) be the number of files and \(S\) the total number of bytes read during hashing.

Operation	Complexity
Scanning files	\(O(n)\)
Grouping by size	\(O(n)\) expected

Hash calculation	\(O(S)\)
Overall time	\(O(n+S)\) expected
Auxiliary space	\(O(n)\) approximately

4.5 Summary

The algorithm uses file-size grouping and SHA-256 hashing to detect exact duplicate files. Its expected time complexity is \(O(n+S)\), and its auxiliary space complexity is approximately \(O(n)\).

CHAPTER 5: ALGORITHM AND METHODOLOGY

5.1 Basic Concept

The project uses a two-stage comparison method.

First, it compares file sizes. Files with different sizes cannot contain exactly the same complete byte sequence, so they do not need to be compared further.

Second, it calculates SHA-256 hashes for files of the same size. Files with matching hashes are grouped as duplicate candidates.

This approach avoids hashing every file unnecessarily.

5.2 Algorithm

Algorithm: Duplicate File Detection

Input: A selected directory.

Output: Groups of duplicate file candidates.

1. **Begin.**
2. **Read the selected directory.**
3. **Recursively visit the directory and its subdirectories.**
4. **Collect the path and size of each accessible file.**
5. **Group files according to their sizes.**
6. **Ignore size groups containing only one file.**
7. **Calculate the SHA-256 hash of each file in the remaining groups.**
8. **Group files having matching hashes.**
9. **Keep groups containing two or more files.**
10. **Display the duplicate groups.**
11. **Allow the user to export results or quarantine selected files.**
12. **End.**

5.3 Pseudocode

START

Select directory

FOR each accessible file in directory and subdirectories

```
    Read file path
    Read file size
    Add file information to size group
END FOR

FOR each size group
    IF group contains more than one file
        FOR each file in group
            Calculate SHA-256 hash
            Add file to hash group
        END FOR

        FOR each hash group
            IF group contains more than one file
                Record duplicate candidates
            END IF
        END FOR
    END IF
END FOR

Display duplicate candidates

IF user requests CSV export
```

Export duplicate information

END IF

IF user selects files for quarantine

Move selected files to quarantine folder

END IF

END

5.4 SHA-256 Hashing

A hash function transforms input data into a fixed-length value. SHA-256 produces a 256-bit hash, commonly represented as 64 hexadecimal characters.

For example, two files with identical contents will normally produce the same SHA-256 hash, even if their filenames or folder locations differ.

The application reads file data in chunks to avoid loading an entire large file into memory at once.

A hash match is treated as strong evidence that the file contents are identical. However, hash equality is not a mathematical proof that collisions are impossible.

5.5 Time Complexity

Let:

- **\(n\) = number of files.**
- **\(B\) = total number of bytes read while calculating hashes.**
- **\(k\) = number of files within a same-size candidate group.**

Collecting metadata requires approximately \(O(n)\) work, excluding file-system overhead.

Hashing requires time proportional to the amount of data read, approximately \(O(B)\). Grouping using hash-based dictionaries is typically expected to take linear time in the number of processed files.

The overall time complexity is approximately:

\[O(n+B) \]

This is an expected practical bound under ordinary hash-table assumptions. File-system operations, disk speed, and repeated processing can affect actual performance.

5.6 Space Complexity

The application stores file paths, sizes, and hash information while processing the scan.

The metadata storage is approximately \(O(n)\), excluding the memory used by the graphical interface and temporary buffers.

The file contents are read in chunks, so the application does not need to store every file entirely in memory.

CHAPTER 6: IMPLEMENTATION

6.1 Implementation Overview

The Duplicate File Finder is implemented as a Python desktop application. The program combines a graphical interface with file-scanning and duplicate-detection functions.

The user selects a directory and starts scanning. The program processes the directory in the background and reports the results in the interface.

The application can export results to CSV and move selected duplicate files into a quarantine folder.

6.2 Folder Selection

The application uses a folder-selection dialog to let the user choose a directory. This makes it easier to use because the user does not need to manually type the full path.

After a folder is selected, the application uses that directory as the starting point for the scan.

6.3 File Scanning

The scanner visits the selected directory and its subdirectories. It collects the paths and sizes of accessible files.

Some files may not be accessible because of permissions or because they are in use. These errors should be handled so that one inaccessible file does not unnecessarily terminate the entire scan.

6.4 Duplicate Detection

The application groups files by size before comparing their contents. This reduces the number of files requiring hash calculation.

For files with the same size, the program calculates a SHA-256 hash. Files with matching hashes are grouped together as duplicate candidates.

6.5 Graphical User Interface

Tkinter is used to create the graphical interface.

The interface provides controls for selecting a folder, starting and cancelling a scan, viewing results, exporting a CSV report, and quarantining selected files.

The graphical interface is intended to make the program understandable to users who do not have programming knowledge.

6.6 CSV Export

The CSV export feature saves information about duplicate groups in a tabular format.

The report can contain file paths, file sizes, and duplicate-group information, depending on the fields included by the implementation.

CSV files are useful because they can be opened in spreadsheet applications for further review.

6.7 Quarantine Feature

The quarantine feature moves selected files to a designated quarantine folder instead of permanently deleting them.

This provides an additional opportunity to review the files before deciding whether they are still required.

Users should review the selected files carefully and keep backups of important data. Moving files can affect applications that depend on their original locations.

6.8 Exception Handling

Exception handling is used to manage errors that may occur during scanning, reading, exporting, or moving files.

Examples include:

- **Permission-denied errors.**
- **Missing or moved files.**
- **Files that cannot be read.**
- **Invalid destination paths.**
- **Errors while writing CSV reports.**

Appropriate error handling improves the usability and reliability of the application.

6.9 Main Python Modules

Module	Use in the Project
tkinter	Creates the graphical interface
os	Supports file and directory operations

pathlib	**Handles file-system paths**
hashlib	**Calculates SHA-256 hashes**
csv	**Writes CSV reports**
threading	**Performs scanning in a background thread**
queue	**Transfers progress and results to the interface**
shutil	**Supports moving files into quarantine**

These are Python standard-library modules; the project does not require third-party packages for its core functionality.

CHAPTER 7: TESTING AND RESULTS

7.1 Testing

Testing is necessary to verify that the application works correctly under different conditions.

The application should be tested with files containing identical data, files with different contents, empty directories, nested folders, and situations where file access is restricted.

The following table describes suggested test cases. Their actual outcomes should be filled in after running the program.

7.2 Test Cases

Test Case	Input or Condition	Expected Result	Actual Result
TC01	Two files with identical contents and different names	Files identified as duplicate candidates	To be filled after testing
TC02	Two files with different contents but the same size	Files should not be grouped as duplicates if their hashes differ	To be filled after testing

TC03	**Two files with different sizes**	**Files should not be identified as exact duplicates**	**To be filled after testing**
TC04	**Folder containing no files**	**Scan completes with no duplicate groups**	**To be filled after testing**
TC05	**Folder containing subfolders**	**Files in subfolders are scanned**	**To be filled after testing**
TC06	**File that cannot be accessed**	**Error is handled without unnecessarily stopping the entire scan**	**To be filled after testing**
TC07	**Export duplicate results to CSV**	**A CSV report is created**	**To be filled after testing**
TC08	**Quarantine selected duplicate files**	**Selected files are moved to the quarantine location**	**To be filled after testing**

7.3 Expected Results

The expected output of the application is a list of groups containing files with matching sizes and SHA-256 hashes.

For each duplicate group, the application should display relevant information such as file paths and file sizes. The application may also provide a summary of the number of duplicate files and the storage space that could potentially be recovered.

The results depend on the files present in the selected directory and the success of the scan.

7.4 Result Analysis

After testing, the actual results should be recorded in the test-case table. If any test fails, the issue should be investigated and corrected.

For example, if two files have the same contents but different filenames, the program should identify them as duplicate candidates. If two files have the same size but different contents, their hashes should normally differ and they should not be grouped together.

No measured performance figures or experimental results are claimed in this report because they must be obtained by running the application.

CHAPTER 8: ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

8.1 Advantages

1. **Saves time: Automates duplicate file detection.**
2. **Improves file management: Helps users identify repeated files.**
3. **Reduces unnecessary comparisons: Uses file sizes as an initial filter.**
4. **Content-based comparison: Uses SHA-256 instead of relying only on filenames.**
5. **Easy to use: Provides a graphical user interface.**
6. **Report generation: Exports duplicate information to CSV.**
7. **Safer file handling: Provides a quarantine option instead of immediate permanent deletion.**
8. **No additional package installation: Uses Python's standard library.**

8.2 Limitations

1. **Scanning large folders may take time.**
2. **Hashing large files requires reading their contents.**
3. **Some files may be skipped or reported as inaccessible because of permissions or other errors.**
4. **The application depends on the file-system access available to the current user.**

5. **Moving files into quarantine may affect software that expects those files at their original locations.**
6. **The program is designed to identify identical file contents, not visually similar photographs or nearly identical documents.**
7. **Actual performance depends on storage speed, file sizes, and the number of files.**

8.3 Future Scope

The project can be improved in several ways:

- **Add a restore feature for quarantined files.**
- **Add filters for file extensions and minimum file sizes.**
- **Provide more detailed scan reports.**
- **Add options to exclude selected directories.**
- **Improve progress reporting for large scans.**
- **Add a preview option for images.**
- **Provide a confirmation screen before moving files.**
- **Add more advanced duplicate detection for visually similar images.**
- **Add automated tests and performance benchmarks.**
- **Improve the interface with additional sorting and filtering options.**

CHAPTER 9: CONCLUSION

The Duplicate File Finder project demonstrates how Python can be used to solve a practical file-management problem. The application scans a selected directory, collects file information, compares file sizes, and uses SHA-256 hashing to identify duplicate file candidates.

The project combines several Python concepts, including file handling, functions, data structures, hashing, exception handling, threading, and graphical user interface development.

The CSV export feature allows users to save duplicate information for later review, while the quarantine feature provides a safer alternative to immediately deleting files.

Developing this project improves understanding of Python programming and demonstrates how basic programming concepts can be combined to create a useful desktop application.

In the future, the application can be enhanced with additional safety features, more detailed reports, improved testing, and support for identifying visually similar images.

REFERENCES

1. **Python Software Foundation, Python Documentation.** https://docs.python.org/3/
2. **Python Software Foundation, Tkinter — Python Interface to Tcl/Tk.** https://docs.python.org/3/library/tkinter.html
3. **Python Software Foundation, hashlib — Secure Hashes and Message Digests.** https://docs.python.org/3/library/hashlib.html
4. **Python Software Foundation, csv — CSV File Reading and Writing.** https://docs.python.org/3/library/csv.html
5. **Python Software Foundation, threading — Thread-Based Parallelism.** https://docs.python.org/3/library/threading.html
6. **National Institute of Standards and Technology (NIST), Secure Hash Standard (SHS).** https://csrc.nist.gov/pubs/fips/180-4/upd1/final